Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To recommend an item to the user using collaborative-based filtering approach	
Experiment No: 12	**Date: 01-04-2026**	**Enrolment No: 92410133046**

Aim: To recommend an item to the user using collaborative-based filtering approach

IDE: Google Colab

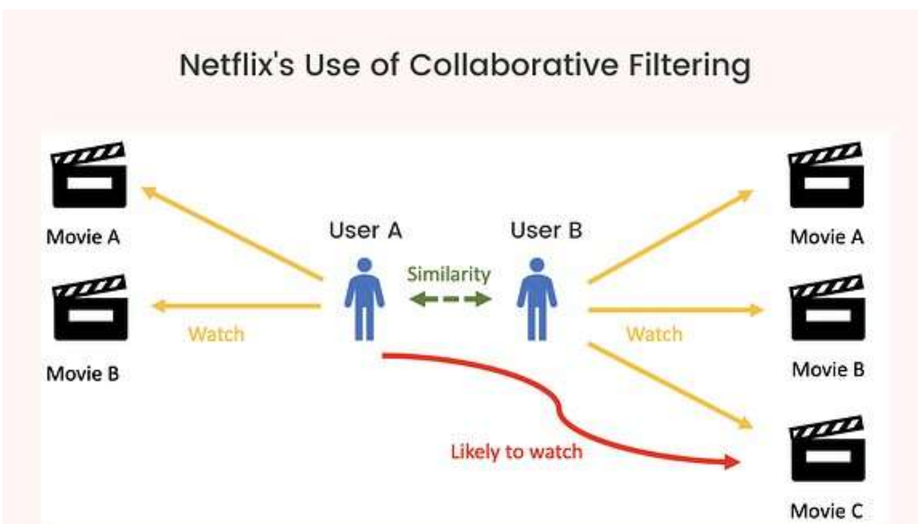
Theory:

There are a lot of applications where websites collect data from their users and use that data to predict the likes and dislikes of their users. This allows them to recommend the content that they like. Recommender systems are a way of suggesting or similar items and ideas to a user's specific way of thinking.

Recommender System is different types:

- **Collaborative Filtering:** Collaborative Filtering recommends items based on similarity measures between users and/or items. The basic assumption behind the algorithm is that users with similar interests have common preferences.
- **Content-Based Recommendation:** It is supervised machine learning used to induce a classifier to discriminate between interesting and uninteresting items for the user.

In Collaborative Filtering, we tend to find similar users and recommend what similar users like. In this type of recommendation system, we don't use the features of the item to recommend it, rather we classify the users into clusters of similar types and recommend each user according to the preference of its cluster.



Pre Lab Exercise:

1. **When can you use collaborative-based recommendation system approach?**

   Collaborative-based recommendation systems come into play when there's user interaction data, e.g. ratings clicks, or purchase history. This method is fit when there are a lot of users on the platform, and you can use their behavior to identify similarities between users or items. It's a standard approach in platforms where recommendations rely on the collective preferences of users -- e.g. movie product, or music recommendation platforms. It is most effective when there is enough historical data and user activity available.

2. **Write advantages of collaborative -based filtering approach**

   Collaborative filtering relies on the behavior of like-minded users for making recommendations, thus enabling the system to suggest a wide variety of items, including those that a user might not discover on their own, as well as unexpected ones. At the same time, it doesn't require exhaustive information about items in order to work, which makes it efficient even when item attributes are scant or absent. Besides that, as the system accumulates new user data, it becomes more and more precise. Moreover, by looking at patterns in user interactions, it can discover and understand highly sophisticated user tastes.

3. **Write disadvantages of collaborative -based filtering approach**

   Collaborative filtering is notorious for its cold start problem, which means that new users or items cannot be suggested well as the system does not have sufficient data on them. Generally, it also loses to the problem of data sparsity, because users usually interact with a small fraction of items only, hence it is hard to establish dependable similarities. It is largely reliant on user data, which can be a concern from a privacy perspective. In addition, it could require a lot of computation for large datasets and may even encounter issues with scaling when the number of users and items grows.

Program (Code):

To be attached with

```
# @title


import pandas as pd
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
ratings = pd.read_csv("/content/sample_data/ratings.csv")
# ================= USER-ITEM MATRIX =================
user_item_matrix = ratings.pivot_table(
    index='userId',
    columns='movieId',
    values='rating'
).fillna(0)
# ================= USER SIMILARITY =================
user_similarity = cosine_similarity(user_item_matrix)
user_similarity_df = pd.DataFrame(
    user_similarity,
    index=user_item_matrix.index,
    columns=user_item_matrix.index
)
# ================= RECOMMEND FUNCTION =================
def recommend_movies(user_id, top_n=5):
    if user_id not in user_item_matrix.index:
        return "User not found!"

    # Similar users
    similar_users = user_similarity_df[user_id].sort_values(ascending=False)[1:6]

    # Weighted ratings
    weighted_ratings = np.zeros(user_item_matrix.shape[1])

    for sim_user, score in similar_users.items():
        weighted_ratings += score * user_item_matrix.loc[sim_user].values
```

```
    # Normalize
    weighted_ratings /= similar_users.sum()

    # Get unseen movies
    user_rated = user_item_matrix.loc[user_id]
    unseen_movies = user_rated[user_rated == 0].index

    scores = list(zip(unseen_movies,
weighted_ratings[user_item_matrix.columns.get_indexer(unseen_movies)]))
    scores = sorted(scores, key=lambda x: x[1], reverse=True)

    recommended_ids = [movie for movie, _ in scores[:top_n]]

    return recommended_ids
print(recommend_movies(user_id=20))
```

Results:

To be attached with

[1, 2571, 377, 6377, 480]

Observation:

The code snippet provided outlines the working of a user-based collaborative filtering recommendation engine which relies on users' rating data. First of all, the dataset with user-movie ratings is brought in and converted into a user-item interaction matrix, so that the rows correspond to users, the columns to movies, and the entries to ratings. The missing values are replaced with zero to mark the absence of interaction.

The next step involves calculating user-to-user similarity through cosine similarity. With this, we get a similarity measure of users based on how they have rated the movies. A similarity matrix is created where each user is linked to all other users through similarity values.

The recommending procedure recommend_movies() receives a user ID and locates the most similar users (top neighbors) by arranging similarity scores in descending order. Then, it figures out weighted ratings by aggregating ratings of these similar users, where each rating is adjusted by the similarity score. This gives a forecasted liking value for each movie.

After that, the program removes from the recommendation list those movies which the user has already rated and works only with the movies that are new to the user. These unviewed titles are sorted according to their estimated scores and the best N movies are picked as recommendations.

Post Lab Exercise:

1. **Are you satisfied with the results/ratings shown by your designed model? If yes, justify. If no, describe the probable reason for under performance of the system.**

The level of fulfillment with results from the planned collaborative filtering model rests on the caliber of personalized suggestions it generates. Usually, the output can be partly agreeable however they are not entirely perfect.

When the model recognizes the likings of users and suggests similar movies to them, it indicates that the identification of user behavior patterns has been done accurately. So if the suggested movies fit the user's preferences, the system can be seen as a good one since it effectively makes use of user similarity and rating data for making predictions.

Nevertheless, if the suggestions are off the mark or appear to be of no interest, the system may be inadequately performing. It can be for a multitude of reasons. For one, data sparsity - when users have rated only a handful of movies each - will cause similarity computations to be weak. The second problem is that of cold starts, which means new users or items lack sufficient data for reliable recommendation generation. Moreover, resorting only to simple cosine similarity without employing advanced methods may not be able to provide an accurate modeling. Also, the system totally ignores user context or content-based features which further dampen the level of recommendation quality.

2. Recommend top-10 "movies" using MovieLens 100K dataset (Link: https://grouplens.org/datasets/movielens/100k/). Attach the code with output. Also, write your observation for the same.